chapter

6

PIPELINING

CHAPTER OBJECTIVES

In this chapter you will learn about:

- Pipelining as a means for improving performance by overlapping the execution of machine instructions
- Hazards that limit performance gains in pipelined processors and means for mitigating their effect
- Hardware and software implications of pipelining
- Influence of pipelining on instruction set design
- Superscalar processors

Chapter 5 introduced the organization of a processor for executing instructions one at a time. In this chapter, we discuss the concept of pipelining, which overlaps the execution of successive instructions to achieve high performance. We begin by explaining the basics of pipelining and how it can lead to improved performance. Then we examine hazards that cause performance degradation and techniques to alleviate their effect on performance. We discuss the role of *optimizing compilers*, which rearrange the sequence of instructions to maximize the benefits of pipelined execution. For further performance improvement, we also consider replicating hardware units in a *superscalar* processor so that multiple pipelines can operate concurrently.

6.1 BASIC CONCEPT—THE IDEAL CASE

The speed of execution of programs is influenced by many factors. One way to improve performance is to use faster circuit technology to implement the processor and the main memory. Another possibility is to arrange the hardware so that more than one operation can be performed at the same time. In this way, the number of operations performed per second is increased, even though the time needed to perform any one operation is not changed.

Pipelining is a particularly effective way of organizing concurrent activity in a computer system. The basic idea is very simple. It is frequently encountered in manufacturing plants, where pipelining is commonly known as an assembly-line operation. Readers are undoubtedly familiar with the assembly line used in automobile manufacturing. The first station in an assembly line may prepare the automobile chassis, the next station adds the body, the next one installs the engine, and so on. While one group of workers is installing the engine on one automobile, another group is fitting a body on the chassis of a second automobile, and yet another group is preparing a new chassis for a third automobile. Although it may take hours or days to complete one automobile, the assembly-line operation makes it possible to have a new automobile rolling off the end of the assembly line every few minutes.

Consider how the idea of pipelining can be used in a computer. The five-stage processor organization in Figure 5.7 and the corresponding datapath in Figure 5.8 allow instructions to be fetched and executed one at a time. It takes five clock cycles to complete the execution of each instruction. Rather than wait until each instruction is completed, instructions can be fetched and executed in a pipelined manner, as shown in Figure 6.1. The five stages corresponding to those in Figure 5.7 are labeled as Fetch, Decode, Compute, Memory, and Write. Instruction I_j is fetched in the first cycle and moves through the remaining stages in the following cycles. In the second cycle, instruction I_{j+1} is fetched while instruction I_j is in the Decode stage where its operands are also read from the register file. In the third cycle, instruction I_{j+2} is fetched while instruction I_{j+1} is in the Decode stage and instruction I_j is in the Compute stage where an arithmetic or logic operation is performed on its operands. Ideally, this overlapping pattern of execution would be possible for all instructions. Although any one instruction takes five cycles to complete its execution, instructions are completed at the rate of one per cycle.

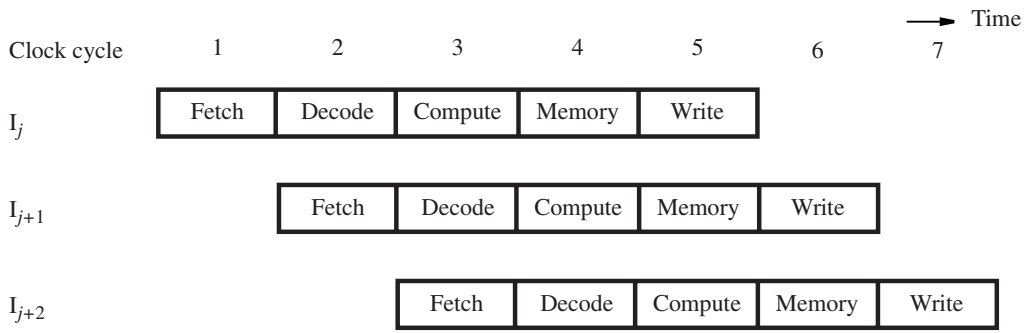


Figure 6.1 Pipelined execution—the ideal case.

6.2 PIPELINE ORGANIZATION

Figure 6.2 indicates how the five-stage organization in Figures 5.7 and 5.8 can be pipelined. In the first stage of the pipeline, the program counter (PC) is used to fetch a new instruction. As other instructions are fetched, execution proceeds through successive stages. At any given time, each stage of the pipeline is processing a different instruction. Information such as register addresses, immediate data, and the operations to be performed must be carried through the pipeline as each instruction proceeds from one stage to the next. This information is held in *interstage buffers*. These include registers RA, RB, RM, RY, and RZ in Figure 5.8, the IR and PC-Temp registers in Figures 5.9 and 5.10, and additional storage. The interstage buffers are used as follows:

- Interstage buffer B1 feeds the Decode stage with a newly-fetched instruction.
- Interstage buffer B2 feeds the Compute stage with the two operands read from the register file, the source/destination register identifiers, the immediate value derived from the instruction, the incremented PC value used as the return address for a subroutine call, and the settings of control signals determined by the instruction decoder. The settings for control signals move through the pipeline to determine the ALU operation, the memory operation, and a possible write into the register file.
- Interstage buffer B3 holds the result of the ALU operation, which may be data to be written into the register file or an address that feeds the Memory stage. In the case of a write access to memory, buffer B3 holds the data to be written. These data were read from the register file in the Decode stage. The buffer also holds the incremented PC value passed from the previous stage, in case it is needed as the return address for a subroutine-call instruction.
- Interstage buffer B4 feeds the Write stage with a value to be written into the register file. This value may be the ALU result from the Compute stage, the result of the Memory access stage, or the incremented PC value that is used as the return address for a subroutine-call instruction.

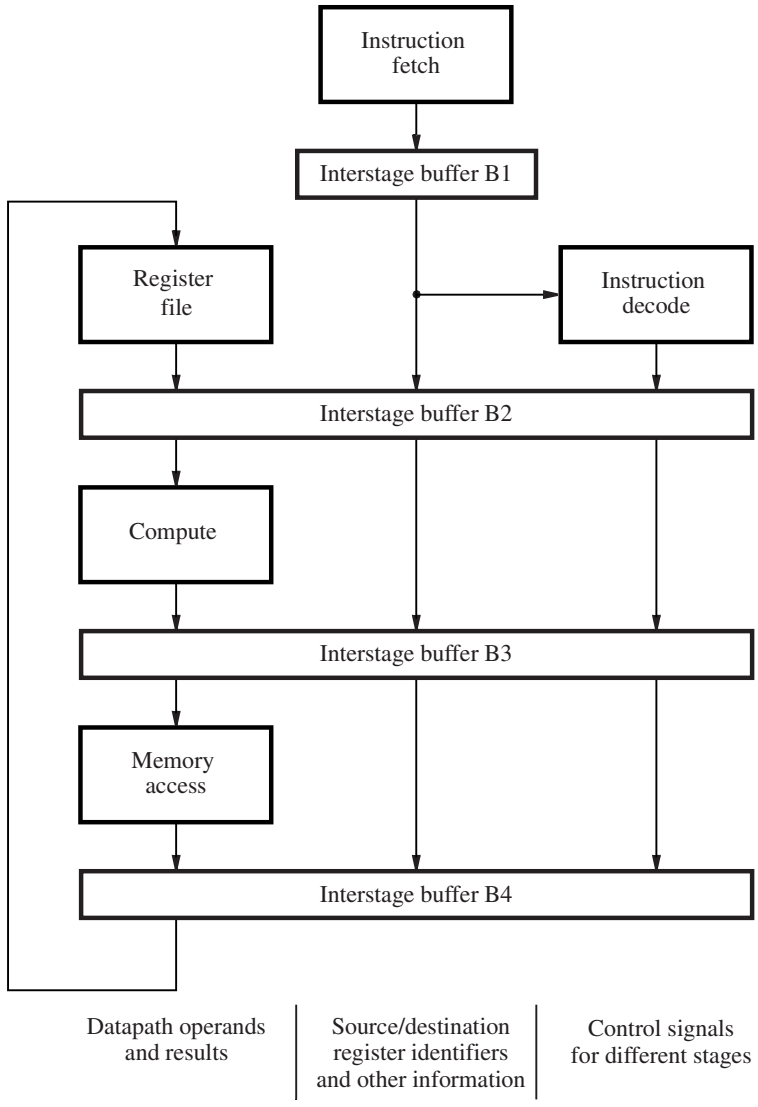


Figure 6.2 A five-stage pipeline.

6.3 PIPELINING ISSUES

Figure 6.1 depicts the ideal overlap of three successive instructions. But, there are times when it is not possible to have a new instruction enter the pipeline in every cycle. Consider the case of two instructions, I_j and I_{j+1}, where the destination register for instruction I_j is a source register for instruction I_{j+1}. The result of instruction I_j is not written into the

register file until cycle 5, but it is needed earlier in cycle 3 when the source operand is read for instruction I_{j+1}. If execution proceeds as shown in Figure 6.1, the result of instruction I_{j+1} would be incorrect because the arithmetic operation would be performed using the old value of the register in question. To obtain the correct result, it is necessary to wait until the new value is written into the register by instruction I_j. Hence, instruction I_{j+1} cannot read its operand until cycle 6, which means it must be *stalled* in the Decode stage for three cycles. While instruction I_{j+1} is stalled, instruction I_{j+2} and all subsequent instructions are similarly delayed. New instructions cannot enter the pipeline, and the total execution time is increased.

Any condition that causes the pipeline to stall is called a *hazard*. We have just described an example of a *data hazard*, where the value of a source operand of an instruction is not available when needed. Other hazards arise from memory delays, branch instructions, and resource limitations. The next several sections describe these hazards in more detail, along with techniques to mitigate their impact on performance.

6.4 DATA DEPENDENCIES

Consider the two instructions in Figure 6.3:

```
Add       R2, R3, #100
Subtract  R9, R2, #30
```

The destination register R2 for the Add instruction is a source register for the Subtract instruction. There is a *data dependency* between these two instructions, because register R2 carries data from the first instruction to the second. Pipelined execution of these two instructions is depicted in Figure 6.3. The Subtract instruction is stalled for three cycles to delay reading register R2 until cycle 6 when the new value becomes available.

We now explain the stall in more detail. The control circuit must first recognize the data dependency when it decodes the Subtract instruction in cycle 3 by comparing its source register identifier from interstage buffer B1 with the destination register identifier of the Add instruction that is held in interstage buffer B2. Then, the Subtract instruction must be held in interstage buffer B1 during cycles 3 to 5. Meanwhile, the Add instruction proceeds through the remaining pipeline stages. In cycles 3 to 5, as the Add instruction moves ahead, control

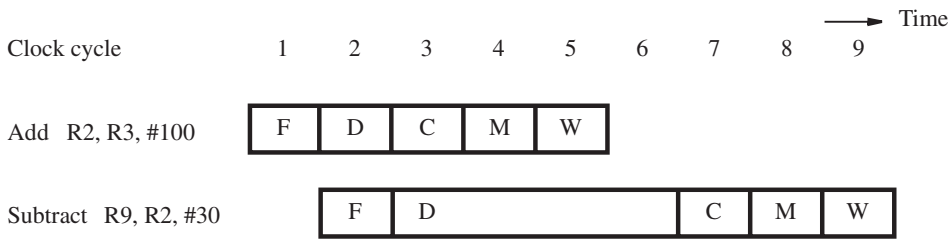


Figure 6.3 Pipeline stall due to data dependency.

signals can be set in interstage buffer B2 for an implicit NOP (No-operation) instruction that does not modify the memory or the register file. Each NOP creates one clock cycle of idle time, called a *bubble*, as it passes through the Compute, Memory, and Write stages to the end of the pipeline.

6.4.1 OPERAND FORWARDING

Pipeline stalls due to data dependencies can be alleviated through the use of *operand forwarding*. Consider the pair of instructions discussed above, where the pipeline is stalled for three cycles to enable the Subtract instruction to use the new value in register R2. The desired value is actually available at the end of cycle 3, when the ALU completes the operation for the Add instruction. This value is loaded into register RZ in Figure 5.8, which is a part of interstage buffer B3. Rather than stall the Subtract instruction, the hardware can *forward* the value from register RZ to where it is needed in cycle 4, which is the ALU input. Figure 6.4 shows pipelined execution when forwarding is implemented. The arrow shows that the ALU result from cycle 3 is used as an input to the ALU in cycle 4.

Figure 6.5 shows the modification needed in the datapath of Figure 5.8 to make this forwarding possible. A new multiplexer, MuxA, is inserted before input InA of the ALU, and the existing multiplexer MuxB is expanded with another input. The multiplexers select either a value read from the register file in the normal manner, or the value available in register RZ.

Forwarding can also be extended to a result in register RY in Figure 5.8. This would handle a data dependency such as the one involving register R2 in the following sequence of instructions:

```
Add       R2, R3, #100
Or        R4, R5, R6
Subtract  R9, R2, #30
```

When the Subtract instruction is in the Compute stage of the pipeline, the Or instruction is in the Memory stage (where no operation is performed), and the Add instruction is in the Write stage. The new value of register R2 generated by the Add instruction is now in register RY. Forwarding this value from register RY to ALU input InA makes it possible

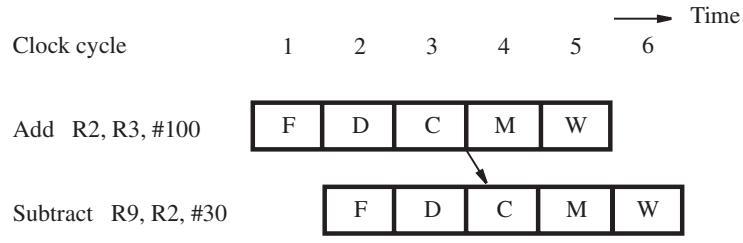


Figure 6.4 Avoiding a stall by using operand forwarding.

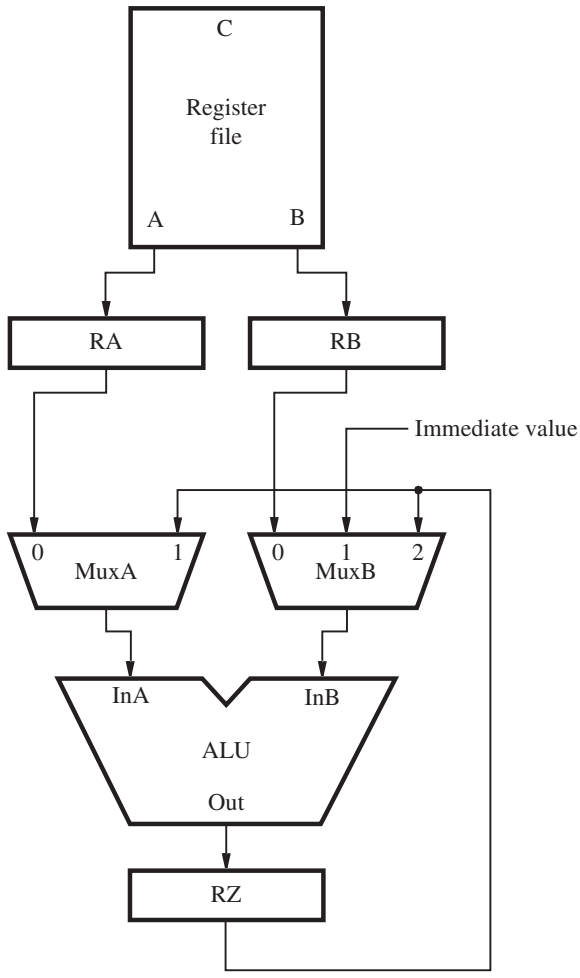


Figure 6.5 Modification of the datapath of Figure 5.8 to support data forwarding from register RZ to the ALU inputs.

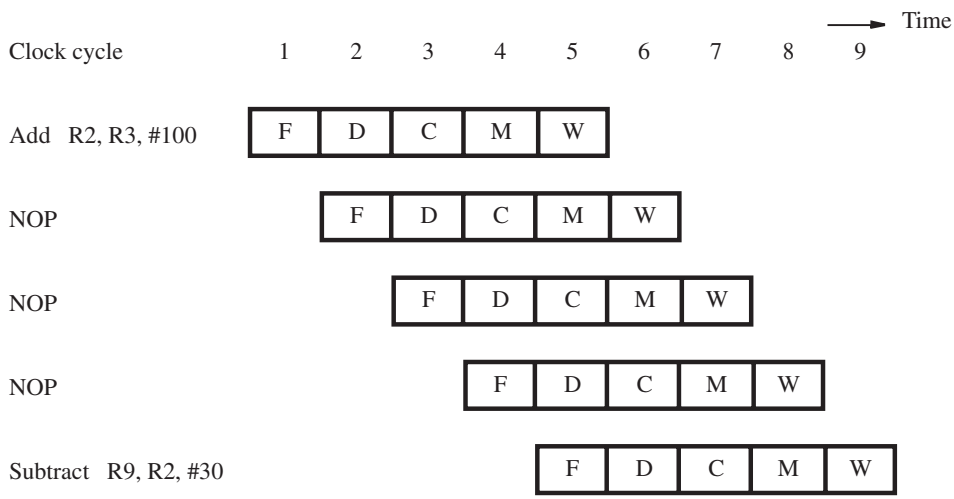
to avoid stalling the pipeline. MuxA requires another input for the value of RY. Similarly, MuxB is extended with another input.

6.4.2 HANDLING DATA DEPENDENCIES IN SOFTWARE

Figures 6.3 and 6.4 show how data dependencies may be handled by the processor hardware, either by stalling the pipeline or by forwarding data. An alternative approach is to leave the task of detecting data dependencies and dealing with them to the compiler. When the

```
Add       R2, R3, #100
NOP
NOP
NOP
Subtract  R9, R2, #30
```

(a) Insertion of NOP instructions for a data dependency



(b) Pipelined execution of instructions

Figure 6.6 Using NOP instructions to handle a data dependency in software.

compiler identifies a data dependency between two successive instructions I_j and I_{j+1}, it can insert three explicit NOP (No-operation) instructions between them. The NOPs introduce the necessary delay to enable instruction I_{j+1} to read the new value from the register file after it is written. For the instructions in Figure 6.4, the compiler would generate the instruction sequence in Figure 6.6*a*. Figure 6.6*b* shows that the three NOP instructions have the same effect on execution time as the stall in Figure 6.3.

Requiring the compiler to identify dependencies and insert NOP instructions simplifies the hardware implementation of the pipeline. However, the code size increases, and the execution time is not reduced as it would be with operand forwarding. The compiler can attempt to *optimize* the code to improve performance and reduce the code size by reordering instructions to move useful instructions into the NOP slots. In doing so, the compiler must consider data dependencies between instructions, which constrain the extent to which the NOP slots can be usefully filled.

6.5 MEMORY DELAYS

Delays arising from memory accesses are another cause of pipeline stalls. For example, a Load instruction may require more than one clock cycle to obtain its operand from memory. This may occur because the requested instruction or data are not found in the cache, resulting in a *cache miss*. Figure 6.7 shows the effect of a delay in accessing data in the memory on pipelined execution. A memory access may take ten or more cycles. For simplicity, the figure shows only three cycles. A cache miss causes all subsequent instructions to be delayed. A similar delay can be caused by a cache miss when fetching an instruction.

There is an additional type of memory-related stall that occurs when there is a data dependency involving a Load instruction. Consider the instructions:

```
Load      R2, (R3)
Subtract  R9, R2, #30
```

Assume that the data for the Load instruction is found in the cache, requiring only one cycle to access the operand. The destination register R2 for the Load instruction is a source register for the Subtract instruction. Operand forwarding cannot be done in the same manner as Figure 6.4, because the data read from memory (the cache, in this case) are not available until they are loaded into register RY at the beginning of cycle 5. Therefore, the Subtract instruction must be stalled for one cycle, as shown in Figure 6.8, to delay the ALU operation. The memory operand, which is now in register RY, can be forwarded to the ALU input in cycle 5.

The compiler can eliminate the one-cycle stall for this type of data dependency by reordering instructions to insert a useful instruction between the Load instruction and the instruction that depends on the data read from the memory. The inserted instruction fills the bubble that would otherwise be created. If a useful instruction cannot be found by the compiler, then the hardware introduces the one-cycle stall automatically. If the processor hardware does not deal with dependencies, then the compiler must insert an explicit NOP instruction.

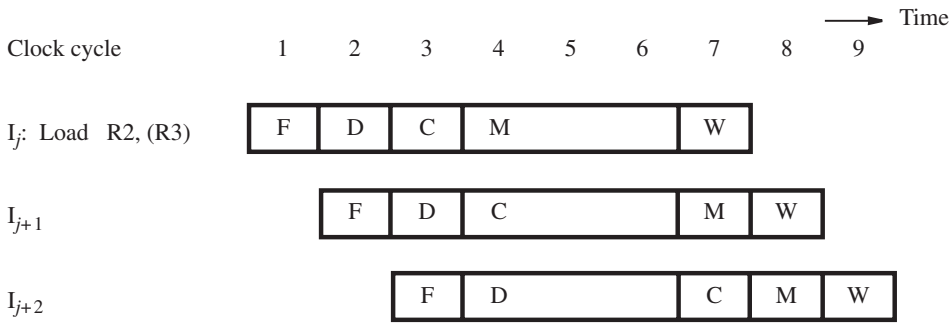


Figure 6.7 Stall caused by a memory access delay for a Load instruction.

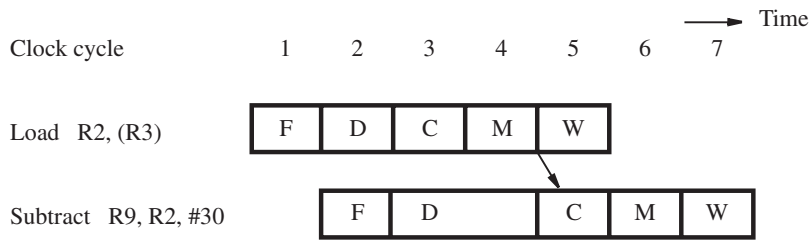


Figure 6.8 Stall needed to enable forwarding for an instruction that follows a Load instruction.

6.6 BRANCH DELAYS

In ideal pipelined execution a new instruction is fetched every cycle, while the preceding instruction is still being decoded. Branch instructions can alter the sequence of execution, but they must first be executed to determine whether and where to branch. We now examine the effect of branch instructions and the techniques that can be used for mitigating their impact on pipelined execution.

6.6.1 UNCONDITIONAL BRANCHES

Figure 6.9 shows the pipelined execution of a sequence of instructions, beginning with an unconditional branch instruction, I_j. The next two instructions, I_{j+1} and I_{j+2}, are stored in successive memory addresses following I_j. The target of the branch is instruction I_k. According to Figure 5.15, the branch instruction is fetched in cycle 1 and decoded in cycle 2, and the target address is computed in cycle 3. Hence, instruction I_k is fetched in cycle 4, after the program counter has been updated with the target address. In pipelined execution, instructions I_{j+1} and I_{j+2} are fetched in cycles 2 and 3, respectively, before the branch instruction is decoded and its target address is known. They must be discarded. The resulting two-cycle delay constitutes a *branch penalty*.

Branch instructions occur frequently. In fact, they represent about 20 percent of the dynamic instruction count of most programs. (The dynamic count is the number of instruction executions, taking into account the fact that some instructions in a program are executed many times, because of loops.) With a two-cycle branch penalty, the relatively high frequency of branch instructions could increase the execution time for a program by as much as 40 percent. Therefore, it is important to find ways to mitigate this impact on performance.

Reducing the branch penalty requires the branch target address to be computed earlier in the pipeline. Rather than wait until the Compute stage, it is possible to determine the target address and update the program counter in the Decode stage. Thus, instruction I_k can be fetched one clock cycle earlier, reducing the branch penalty to one cycle, as shown in Figure 6.10. This time, only one instruction, I_{j+1}, is fetched incorrectly, because the target address is determined in the Decode stage.

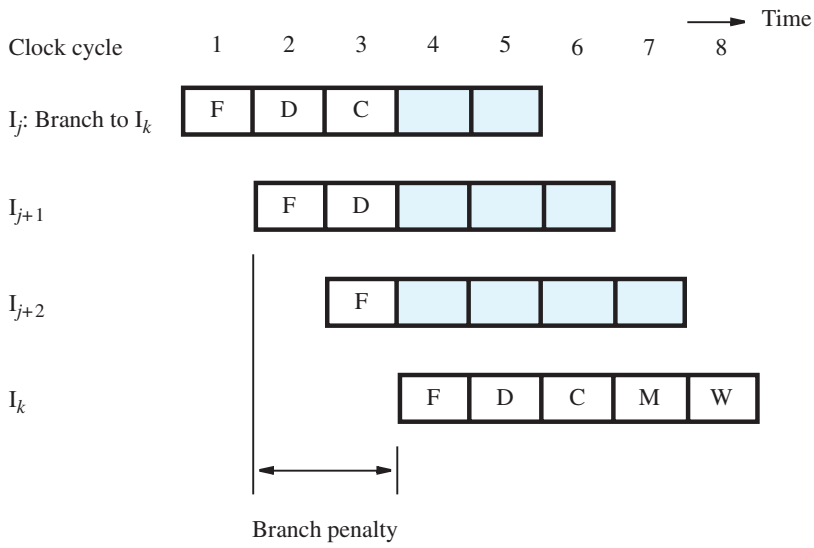


Figure 6.9 Branch penalty when the target address is determined in the Compute stage of the pipeline.

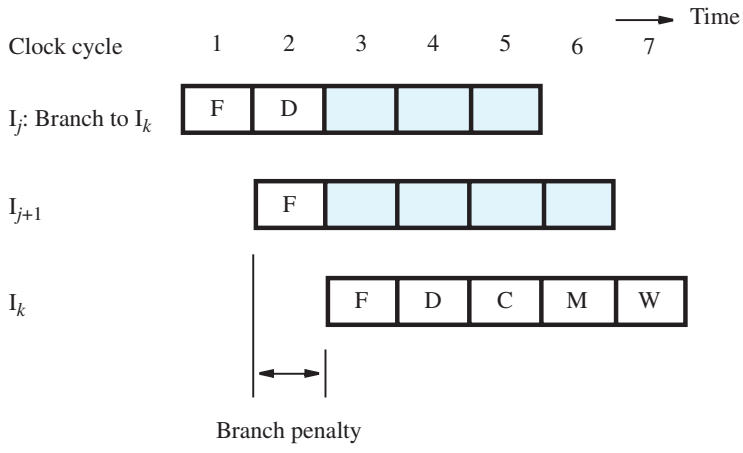


Figure 6.10 Branch penalty when the target address is determined in the Decode stage of the pipeline.

The hardware in Figure 5.10 must be modified to implement this change. The adder in the figure is needed to increment the PC in every cycle. A second adder is needed in the Decode stage to compute a branch target address for every instruction. When the instruction decoder determines that the instruction is indeed a branch instruction, the computed target address will be available before the end of the cycle. It can then be used to fetch the target instruction in the next cycle.

6.6.2 CONDITIONAL BRANCHES

Consider a conditional branch instruction such as

Branch_if_[R5]=[R6] LOOP

The execution steps for this instruction are shown in Figure 5.16. The result of the comparison in the third step determines whether the branch is taken.

For pipelining, the branch condition must be tested as early as possible to limit the branch penalty. We have just described how the target address for an unconditional branch instruction can be determined in the Decode stage. Similarly, the comparator that tests the branch condition can also be moved to the Decode stage, enabling the conditional branch decision to be made at the same time that the target address is determined. In this case, the comparator uses the values from outputs A and B of the register file directly.

Moving the branch decision to the Decode stage ensures a common branch penalty of only one cycle for all branch instructions. In the next two sections, we discuss additional techniques that can be used to further mitigate the effect of branches on execution time.

6.6.3 THE BRANCH DELAY SLOT

Consider the program fragment shown in Figure 6.11*a*. Assume that the branch target address and the branch decision are determined in the Decode stage, at the same time that instruction I_{j+1} is fetched. The branch instruction may cause instruction I_{j+1} to be discarded, after the branch condition is evaluated. If the condition is true, then there is a branch penalty of one cycle before the correct target instruction I_k is fetched. If the condition is false, then instruction I_{j+1} is executed, and there is no penalty. In both of these cases, the instruction immediately following the branch instruction is always fetched. Based on this observation, we describe a technique to reduce the penalty for branch instructions.

The location that follows a branch instruction is called the *branch delay slot*. Rather than conditionally discard the instruction in the delay slot, we can arrange to have the pipeline always execute this instruction, whether or not the branch is taken. The instruction in the delay slot cannot be I_{j+1}, the one that may be discarded depending on the branch condition. Instead, the compiler attempts to find a suitable instruction to occupy the delay slot, one that needs to be executed even when the branch is taken. It can do so by moving one of the instructions preceding the branch instruction to the delay slot. Of course, this can only be done if any data dependencies involving the instruction being moved are preserved. If a useful instruction is found, then there will be no branch penalty. If no useful instruction can be placed in the delay slot because of constraints arising from data dependencies, a NOP must be placed there instead. In this case, there will be a penalty of one cycle whether or not the branch is taken.

For the instructions in Figure 6.11*a*, the Add instruction can safely be moved into the branch delay slot, as shown in Figure 6.11*b*. The Add instruction is always fetched and executed, even if the branch is taken. Instruction I_{j+1} is fetched only if the branch is not taken. Logically, execution proceeds as though the branch instruction were placed after the

```
                Add                 R7, R8, R9
                Branch_if_[R3]=0    TARGET
                I_(j+1)
                ⋮
TARGET:         I_k
```

(a) Original sequence of instructions containing a conditional branch instruction

```
                Branch_if_[R3]=0    TARGET
                Add                 R7, R8, R9
                I_(j+1)
                ⋮
TARGET:         I_k
```

(b) Placing the Add instruction in the branch delay slot where it is always executed

Figure 6.11 Filling the branch delay slot with a useful instruction.

Add instruction. That is, branching takes place one instruction later than where the branch instruction appears in the instruction sequence. This technique is called *delayed branching*.

The effectiveness of delayed branching depends on how often the compiler can reorder instructions to usefully fill the delay slot. Experimental data collected from many programs indicate that the compiler can fill a branch delay slot in 70 percent or more of the cases.

6.6.4 BRANCH PREDICTION

The discussion above shows that making the branch decision in cycle 2 of the execution of a branch instruction reduces the branch penalty. But, even then, the instruction immediately following the branch instruction is still fetched in cycle 2 and may have to be discarded. The decision to fetch this instruction is actually made in cycle 1, when the PC is incremented while the branch instruction itself is being fetched. Thus, to reduce the branch penalty further, the processor needs to anticipate that an instruction being fetched is a branch instruction and *predict* its outcome to determine which instruction should be fetched in cycle 2. In this section, we first describe different methods for branch prediction. Then, we discuss how the prediction is made in cycle 1 while a branch instruction is being fetched.

Static Branch Prediction

The simplest form of branch prediction is to assume that the branch will not be taken and to fetch the next instruction in sequential address order. If the prediction is correct, the fetched instruction is allowed to complete and there is no penalty. However, if it is determined that the branch is to be taken, the instruction that has been fetched is discarded and the correct branch target instruction is fetched. Misprediction incurs the full branch penalty. This simple approach is a form of *static branch prediction*. The same choice (assume not-taken) is used every time a conditional branch is encountered.

If branch outcomes were random, then half of all conditional branches would be taken. In this case, always assuming that branches will not be taken results in a prediction accuracy of 50 percent. However, a backward branch at the end of a loop is taken most of the time. For such a branch, better accuracy can be achieved by predicting that the branch is likely to be taken. Thus, instructions are fetched using the branch target address as soon as it is known. Similarly, for a forward branch at the beginning of a loop, the not-taken prediction leads to good prediction accuracy. The processor can determine the static prediction of taken or not-taken by checking the sign of the branch offset. Alternatively, the machine encoding of a branch instruction may include one bit that indicates whether the branch should be predicted as taken or nor taken. The setting of this bit can be specified by the compiler.

Dynamic Branch Prediction

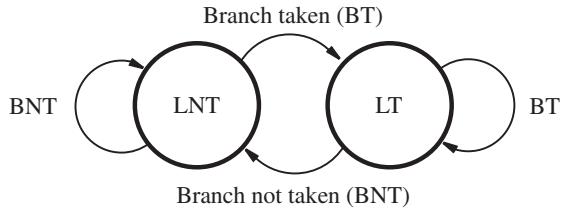
To improve prediction accuracy further, we can use actual branch behavior to influence the prediction, resulting in *dynamic branch prediction*. The processor hardware assesses the likelihood of a given branch being taken by keeping track of branch decisions every time that a branch instruction is executed.

In its simplest form, a dynamic prediction algorithm can use the result of the most recent execution of a branch instruction. The processor assumes that the next time the instruction is executed, the branch decision is likely to be the same as the last time. Hence, the algorithm may be described by the two-state machine in Figure 6.12*a*. The two states are:

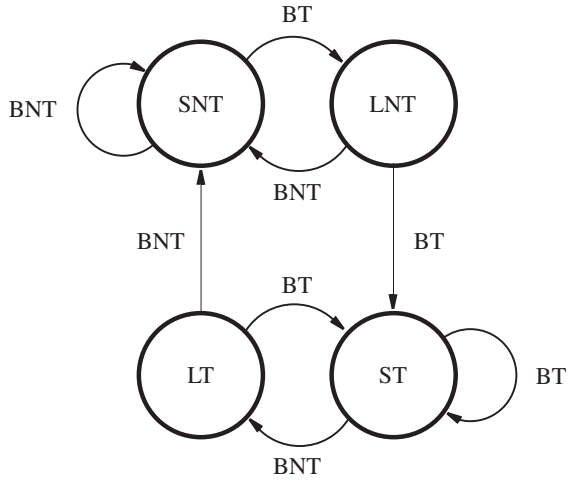
LT - Branch is likely to be taken
LNT - Branch is likely not to be taken

Suppose that the algorithm is started in state LNT. When the branch instruction is executed and the branch is taken, the machine moves to state LT. Otherwise, it remains in state LNT. The next time the same instruction is encountered, the branch is predicted as taken if the state machine is in state LT. Otherwise it is predicted as not taken.

This simple scheme, which requires only a single bit to represent the history of execution for a branch instruction, works well inside program loops. Once a loop is entered, the decision for the branch instruction that controls looping will always be the same except for the last pass through the loop. Hence, each prediction for the branch instruction will be correct except in the last pass. The prediction in the last pass will be incorrect, and the branch history state machine will be changed to the opposite state. Unfortunately, this means that the next time this same loop is entered—and assuming that there will be more than one pass through the loop—the state machine will lead to the wrong prediction for the



(a) A 2-state algorithm



(b) A 4-state algorithm

Figure 6.12 State-machine representation of branch prediction algorithms.

first pass. Thus, repeated execution of the same loop results in mispredictions in the first pass and the last pass.

Better prediction accuracy can be achieved by keeping more information about execution history. An algorithm that uses four states is shown in Figure 6.12*b*. The four states are:

ST	-	Strongly likely to be taken
LT	-	Likely to be taken
LNT	-	Likely not to be taken
SNT	-	Strongly likely not to be taken

Again assume that the state of the algorithm is initially set to LNT. After the branch instruction is executed, and if the branch is actually taken, the state is changed to ST; otherwise, it is changed to SNT. As program execution progresses and the same branch instruction is encountered multiple times, the state of the prediction algorithm changes as shown. The branch is predicted as taken if the state is either ST or LT. Otherwise, the branch is predicted as not taken.

Let us reconsider what happens when executing a program loop. Assume that the branch instruction is at the end of the loop and that the processor sets the initial state of the algorithm to LNT. In the first pass, the prediction (not taken) will be wrong, and hence the state will be changed to ST. In all subsequent passes, the prediction will be correct, except for the last pass. At that time, the state will change to LT. When the loop is entered a second time, the prediction in the first pass will be to take the branch, which will be correct if there is more than one iteration. Thus, repeated execution of the same loop now results in only one misprediction in the last pass.

Branch Target Buffer for Dynamic Prediction

In earlier discussion, we pointed out that the branch target address and the branch decision can both be determined in the Decode stage of the pipeline, which is cycle 2 of instruction execution. The instruction being fetched in the same cycle may or may not be the one that has to be executed after the branch instruction. It may have to be discarded, in which case the correct instruction will be fetched in cycle 3. How can branch prediction be used to obtain better performance?

The key to improving performance is to increase the likelihood that the instruction fetched in cycle 2 is the correct one. This can be achieved only if branch prediction takes place in cycle 1, at the same time that the branch instruction is being fetched. To make this possible, the processor needs to keep more information about the history of execution. The required information is usually stored in a small, fast memory called the *branch target buffer*.

The branch target buffer identifies branch instructions by their addresses. As each branch instruction is executed, the processor records the address of the instruction and the outcome of the branch decision in the buffer. The information is organized in the form of a lookup table, in which each entry includes:

- the address of the branch instruction
- one or two state bits for the branch prediction algorithm
- the branch target address

With this information, the processor is able to identify branch instructions and obtain the corresponding branch prediction state bits based on the address of the instruction being fetched.

Every time the processor fetches a new instruction, it checks the branch target buffer for an entry containing the same instruction address. If an entry with that address is found, this means that the instruction being fetched is a branch instruction. The processor is then able to use the state bits to predict whether that branch is likely to be taken. At the same time, the target address is also obtained. The processor is able to obtain this information as the branch instruction is being fetched in cycle 1. In cycle 2, the processor uses the predicted

outcome of the branch to fetch the next instruction. Of course, it must also determine the actual branch decision and target address to determine whether the predicted values were correct. If they are, execution continues without penalty. Otherwise, the instruction that has just been fetched is discarded, and the correct instruction is fetched in cycle 3. The main value of the branch target buffer is that the state information needed for branch prediction and the target address of a branch instruction are both obtained at the same time the branch instruction is being fetched.

Large programs have many branch instructions. A branch target buffer with enough storage to accommodate information for all of them would be large, and searching it quickly would be difficult. For this reason, the table has a limited size, containing information for only the most recently executed branch instructions. Entries in the table are replaced as other branch instructions are executed. Typically, the table contains on the order of 1024 entries.

6.7 RESOURCE LIMITATIONS

Pipelining enables overlapped execution of instructions, but the pipeline stalls when there are insufficient hardware resources to permit all actions to proceed concurrently. If two instructions need to access the same resource in the same clock cycle, one instruction must be stalled to allow the other instruction to use the resource. This can be prevented by providing additional hardware.

Such stalls can occur in a computer that has a single cache that supports only one access per cycle. If both the Fetch and Memory stages of the pipeline are connected to the cache, then it is not possible for activity in both stages to proceed simultaneously. Normally, the Fetch stage accesses the cache in every cycle. However, this activity must be stalled for one cycle when there is a Load or Store instruction in the Memory stage also needing to access the cache. If 25 percent of all instructions executed are Load or Store instructions, these stalls increase the execution time by 25 percent. Using separate caches for instructions and data allows the Fetch and Memory stages to proceed simultaneously without stalling.

6.8 PERFORMANCE EVALUATION

For a non-pipelined processor, the execution time, T, of a program that has a dynamic instruction count of N is given by

$$T = \frac{N \times S}{R}$$

where S is the average number of clock cycles it takes to fetch and execute one instruction, and R is the clock rate in cycles per second. This is often referred to as the *basic performance equation*. A useful performance indicator is the *instruction throughput*, which is the number of instructions executed per second. For non-pipelined execution, the throughput, P_{np}, is given by

$$P_{np} = \frac{R}{S}$$

The processor presented in Chapter 5 uses five cycles to execute all instructions. Thus, if there are no cache misses, S is equal to 5.

Pipelining improves performance by overlapping the execution of successive instructions, which increases instruction throughput even though an individual instruction is still executed in the same number of cycles. For the five-stage pipeline described in this chapter, each instruction is executed in five cycles, but a new instruction can ideally enter the pipeline every cycle. Thus, in the absence of stalls, S is equal to 1, and the ideal throughput with pipelining is

$$P_p = R$$

A five-stage pipeline can potentially increase the throughput by a factor of five. In general, an n-stage pipeline has the potential to increase throughput n times. Thus, it would appear that the higher the value of n, the larger the performance gain. This leads to two questions:

- How much of this potential increase in instruction throughput can actually be realized in practice?
- What is a good value for n?

Any time a pipeline is stalled or instructions are discarded, the instruction throughput is reduced below its ideal value. Hence, the performance of a pipeline is highly influenced by factors such as stalls due to data dependencies between instructions and penalties due to branches. Cache misses increase the execution time even further. We discuss these issues first, and then we return to the question of how many pipeline stages should be used.

6.8.1 EFFECTS OF STALLS AND PENALTIES

The effects of stalls and penalties have been examined qualitatively in the previous sections. We now consider these effects in quantitative terms.

The five-stage pipeline involves memory-access operations in the Fetch and Memory stages, and ALU operations in the Compute stage. The operations with the longest delay dictate the cycle time, and hence the clock rate R. For a processor that has on-chip caches, memory-access operations have a small delay when the desired instructions or data are found in the cache. The delay through the ALU is likely to be the critical parameter. If this delay is 2 ns, then $R = 500$ MHz, and the ideal pipelined instruction throughput is $P_p = 500$ MIPS (million instructions per second).

Consider a processor with operand forwarding in hardware, as explained in Section 6.4.1. This means that there are no penalties due to data dependencies, except in the case of Load instructions. To evaluate the effect of stalls not related to cache misses, we can consider how often a Load instruction is immediately followed by another instruction that uses the result of the memory access. Section 6.5 explained that a one-cycle stall is necessary in such cases. While ideal pipelined execution has $S = 1$, stalls due to such Load instructions have the effect of increasing S by an amount δ_{stall}. For example, assume that Load instructions constitute 25 percent of the dynamic instruction count, and assume that 40 percent of these Load instructions are followed by a dependent instruction. A one-cycle

stall is needed in such cases. Hence, the increase over the ideal case of $S = 1$ is

$$\delta_{\text{stall}} = 0.25 \times 0.40 \times 1 = 0.10$$

That is, the execution time T is increased by 10 percent, and throughput is reduced to

$$P_p = \frac{R}{1 + \delta_{\text{stall}}} = \frac{R}{1.1} = 0.91R$$

The compiler can improve performance by reducing the number of times that a Load instruction is immediately followed by a dependent instruction. A stall is eliminated each time the compiler can safely move a nearby instruction to a position between the Load instruction and the dependent instruction.

Now, consider the penalties due to mispredicting branches during program execution. When both the branch decision and the branch target address are determined in the Decode stage of the pipeline, the branch penalty is one cycle. Assume that branches constitute 20 percent of the dynamic instruction count of a program, and that the average prediction accuracy for branch instructions is 90 percent. In other words, 10 percent of all branch instructions that are executed incur a one-cycle penalty due to misprediction. The increase in the average number of cycles per instruction due to branch penalties is

$$\delta_{\text{branch\_penalty}} = 0.20 \times 0.10 \times 1 = 0.02$$

High prediction accuracy is beneficial in limiting the adverse impact of this penalty on performance.

The stalls related to Load instructions and the penalties from branch misprediction are independent. Hence, their effect on performance is additive. The sum of δ_{stall} and $\delta_{\text{branch\_penalty}}$ determines the increase in the number of cycles, S, the increase in the execution time, T, and the reduction in the throughput, P_p.

The effect of cache misses on performance can be assessed by considering the frequency of their occurrence. The time to access the slower main memory is a penalty that stalls the pipeline for p_m cycles every time there is a cache miss. A fraction m_i of all instructions that are fetched incur a cache miss. A fraction d of all instructions are Load or Store instructions, and a fraction m_d of these instructions incur a cache miss. The increase over the ideal case of $S = 1$ due to cache misses is

$$\delta_{\text{miss}} = (m_i + d \times m_d) \times p_m$$

Suppose that 5 percent of all fetched instructions incur a cache miss, 30 percent of all instructions executed are Load or Store instructions, and 10 percent of their data-operand accesses incur a cache miss. Assume that the penalty to access the main memory for a cache miss is 10 cycles. The increase over the ideal case of $S = 1$ due to cache misses in this case is given by

$$\delta_{\text{miss}} = (0.05 + 0.30 \times 0.10) \times 10 = 0.8$$

Compared to δ_{stall} for data dependencies and $\delta_{\text{branch\_penalty}}$ for mispredicted branches, the effect of a slow main memory for cache misses is more significant in this example. When all factors are combined, S is increased from the ideal value of 1 to $1 + \delta_{\text{stall}} + \delta_{\text{branch\_penalty}} + \delta_{\text{miss}}$. The contribution of cache misses is often the dominant one.

6.8.2 NUMBER OF PIPELINE STAGES

The fact that an *n*-stage pipeline may increase instruction throughput by a factor of *n* suggests that we should use a large number of stages. However, as the number of pipeline stages increases, there are more instructions being executed concurrently. Consequently, there are more potential dependencies between instructions that may lead to pipeline stalls. Furthermore, the branch penalty may be larger than one cycle if a longer pipeline moves the branch decision to a later stage. For these reasons, the gain in throughput from increasing the value of *n* begins to diminish, and the cost of a deeper pipeline may not be justified.

Another important factor is the inherent delay in the basic operations performed by the processor. The most important among these is the ALU delay. In many processors, the cycle time of the processor clock is chosen such that one ALU operation can be completed in one cycle. Other operations, including accesses to a cache memory, are typically divided into steps that each take about the same time as an ALU operation. Further reductions in the clock cycle time are possible if a pipelined ALU is used. Some recent processor implementations have used twenty or more pipeline stages to aggressively reduce the cycle time. Implementing such long pipelines using modern technology allows for clock rates of several GHz.

6.9 SUPERSCALAR OPERATION

The maximum throughput of a pipelined processor is one instruction per clock cycle. A more aggressive approach is to equip the processor with multiple execution units, each of which may be pipelined, to increase the processor's ability to handle several instructions in parallel. With this arrangement, several instructions start execution in the same clock cycle, but in different execution units, and the processor is said to use *multiple-issue*. Such processors can achieve an instruction execution throughput of more than one instruction per cycle. They are known as *superscalar* processors. Many modern high-performance processors use this approach.

To enable multiple-issue execution, a superscalar processor has a more elaborate *fetch unit* that fetches two or more instructions per cycle before they are needed and places them in an instruction queue. A separate unit, called the *dispatch unit*, takes two or more instructions from the front of the queue, decodes them, and sends them to the appropriate execution units. At the end of the pipeline, another unit is responsible for writing results into the register file. Figure 6.13 shows a superscalar processor with this organization. It incorporates two execution units, one for arithmetic instructions and another for Load and Store instructions. Arithmetic operations normally require only one cycle, hence the first execution unit is simple. Because Load and Store instructions involve an address calculation for the Index mode before each memory access, the Load/Store unit has a two-stage pipeline.

The organization in Figure 6.13 raises some important implications for the register file. An arithmetic instruction and a Load or Store instruction must obtain all their operands from the register file when they are dispatched in the same cycle to the two execution units. The register file must now have four output ports instead of the two output ports needed in

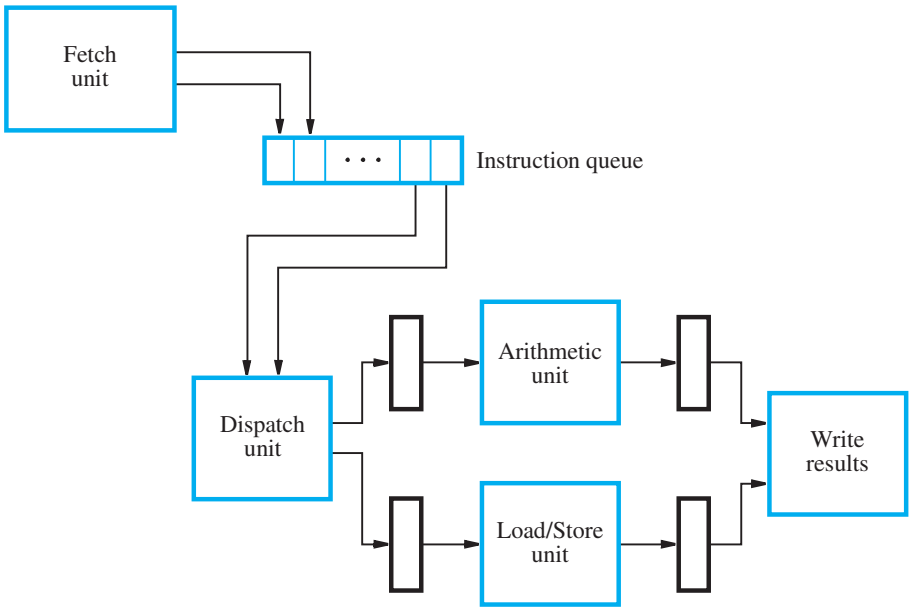


Figure 6.13 A superscalar processor with two execution units.

the simple pipeline. Similarly, an arithmetic instruction and a Load instruction must write their results into the register file when they complete in the same cycle. Thus, the register file must now have two input ports instead of the single input port for the simple pipeline. There is also the potential complication of two instructions completing at the same time with the same destination register for their results. This complication is avoided, if possible, by dispatching the instructions in a manner that prevents its occurrence. Otherwise, one instruction is stalled to ensure that results are written into the destination register in the same order as in the original instruction sequence of the program.

To illustrate superscalar execution in the processor in Figure 6.13, consider the following sequence of instructions:

```
Add       R2, R3, #100
Load      R5, 16(R6)
Subtract  R7, R8, R9
Store     R10, 24(R11)
```

Figure 6.14 shows how these instructions would be executed. The fetch unit fetches two instructions every cycle. The instructions are decoded and their source registers are read in the next cycle. Then, they are dispatched to the arithmetic and Load/Store units. Arithmetic operations can be initiated every cycle. A Load or Store instruction can also be initiated every cycle, because the two-stage pipeline overlaps the address calculation for one Load or Store instruction with the memory access for the preceding Load or Store instruction.

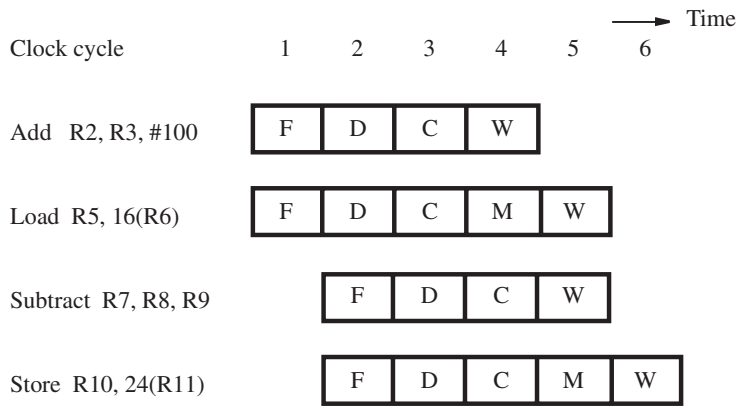


Figure 6.14 An example of instruction flow in the processor of Figure 6.13.

As instructions complete execution in each unit, the register file allows two results to be written in the same cycle because the destination registers are different.

6.9.1 BRANCHES AND DATA DEPENDENCIES

In the absence of any branch instructions and any data dependencies between instructions, throughput is maximized by interleaving instructions that can be dispatched simultaneously to different execution units. However, programs contain branch instructions that change the execution flow, and data dependencies between instructions that impose sequential ordering constraints. A superscalar processor must ensure that instructions are executed in the proper sequence. Furthermore, memory delays due to cache misses may occasionally stall the fetching and dispatching of instructions. As a result, actual throughput is typically below the maximum that is possible. The challenges presented by branch instructions and data dependencies can be addressed with additional hardware. We first consider branch instructions and then consider the issues stemming from data dependencies.

The fetch unit handles branch instructions as it determines which instructions to place in the queue for dispatching. It must determine both the branch decision and the target for each branch instruction. The branch decision may depend on the result of an earlier instruction that is either still queued or newly dispatched. Stalling the fetch unit until the result is available can significantly reduce the throughput and is therefore not a desirable approach. Instead, it is better to employ branch prediction. Since the aim is to achieve high throughput, prediction is also combined with a technique called *speculative execution*. In this technique, subsequent instructions based on an unconfirmed prediction are fetched, dispatched, and possibly executed, but are labeled as being speculative so that they and their results may be discarded if the prediction is incorrect. Additional hardware is required to maintain information about speculatively executed instructions and to ensure that registers or memory locations are not modified until the validity of the prediction is confirmed.

Additional hardware is also needed to ensure that the correct instructions are fetched and dispatched in the event of misprediction.

Data dependencies between instructions impose ordering constraints. A simple approach is to dispatch dependent instructions in sequence to the same execution unit, where their order would be preserved. However, dependent instructions may be dispatched to different execution units. For example, the result of a Load instruction dispatched to the Load/Store unit in Figure 6.13 may be needed by an Add instruction dispatched to the arithmetic unit. Because the units operate independently and because other instructions may have already been dispatched to them, there is no guarantee as to when the result needed by the Add instruction is generated by the Load instruction. A mechanism is needed to ensure that a dependent instruction waits for its operands to become available. When an instruction is dispatched to an execution unit, it is buffered until all necessary results from other instructions have been generated. Such buffers are called *reservation stations*, and they are used to hold information and operands relevant to each dispatched instruction. Results from each execution unit are broadcast to all reservation stations with each result tagged with a register identifier. This enables the reservation stations to recognize a result on which a buffered instruction depends. When there is a matching tag, the hardware copies the result into the reservation station containing the instruction. The control circuit begins the execution of a buffered instruction only when it has all of its operands.

In a superscalar processor using multiple-issue, the detrimental effect of stalls becomes even more pronounced than in a single-issue pipelined processor. The compiler can avoid many stalls through judicious selection and ordering of instructions. For example, for the processor in Figure 6.13, the compiler should strive to interleave arithmetic and memory instructions. This enables the dispatch unit to keep both units busy most of the time.

6.9.2 OUT-OF-ORDER EXECUTION

The instructions in Figure 6.14 are dispatched in the same order as they appear in the program. However, their execution may be completed out of order. For example, the Subtract instruction writes to register R7 in the same cycle as the Load instruction that was fetched earlier writes to register R5. If the memory access for the Load instruction requires more than one cycle to complete, execution of the Subtract instruction would be completed before the Load instruction. Does this type of situation lead to problems?

We have already discussed the issues arising from dependencies among instructions. For example, if an instruction I_{j+1} depends on the result of instruction I_j, the execution of I_{j+1} will be delayed if the result is not available when it is needed. As long as such dependencies are handled correctly, there is no reason to delay the execution of an unrelated instruction. If there is no dependency between a pair of instructions, the order in which execution is completed does not matter.

However, a new complication arises when we consider the possibility of an instruction causing an exception. For example, the Load instruction in Figure 6.14 may attempt an illegal unaligned memory access for a data operand. By the time this illegal operation is recognized, the Subtract instruction that is fetched after the Load instruction may have already modified its destination register. Program execution is now in an inconsistent

state. The instruction that caused the exception in the original sequence is identified, but a succeeding instruction in that sequence has been executed to completion. If such a situation is permitted, the processor is said to have *imprecise exceptions*.

The alternative of *precise exceptions* requires additional hardware. To guarantee a consistent state when exceptions occur, the results of the execution of instructions must be written into the destination locations strictly in program order. This means that we must delay writing into register R7 for the Subtract instruction in Figure 6.14 until after register R5 for the Load instruction has been updated. Either the arithmetic unit in Figure 6.13 must retain the result of the Subtract instruction, or the result must be buffered in a temporary register until preceding instructions have written their results. If an exception occurs during the execution of an instruction, all subsequent instructions and their buffered results are discarded.

It is easier to provide precise exceptions in the case of external interrupts. When an external interrupt is received, the dispatch unit stops reading new instructions from the instruction queue, and the instructions remaining in the queue are discarded. All instructions whose execution is pending continue to completion. At this point, the processor and all its registers are in a consistent state, and interrupt processing can begin.

6.9.3 EXECUTION COMPLETION

To improve performance, an execution unit should be allowed to execute any instructions whose operands are ready in its reservation station. This may lead to out-of-order execution of instructions. However, instructions must be completed in program order to allow precise exceptions. These seemingly conflicting requirements can be resolved if execution is allowed to proceed out of order, but the results are written into temporary registers. The contents of these registers are later transferred to the permanent registers in correct program order. This last step is often called the *commitment* step, because the effect of an instruction cannot be reversed after that point. If an instruction causes an exception, the results of any subsequent instructions that have been executed would still be in temporary registers and can be safely discarded. Results that would normally be written to memory would also be buffered temporarily, and they can be safely discarded as well.

A temporary register that is assigned for the result of an instruction assumes the role of the permanent register whose data it is holding. Its contents are forwarded to any subsequent instruction that refers to the original permanent register during that period. This technique is called *register renaming*. There may be as many temporary registers as there are permanent registers, or there may be fewer temporary registers that are allocated as needed for association with different permanent registers.

When out-of-order execution is allowed, a special control unit is needed to guarantee in-order commitment. This is called the *commitment unit*. It uses a separate queue called the *reorder buffer* to determine which instruction(s) should be committed next. Instructions are entered in the queue strictly in program order as they are dispatched for execution. When an instruction reaches the head of this queue and the execution of that instruction has been completed, the corresponding results are transferred from the temporary registers to the permanent registers and the instruction is removed from the queue. All resources that were assigned to the instruction, including the temporary registers, are released. The instruction

is said to have been *retired* at this point. Because an instruction is retired only when it is at the head of the queue, all instructions that were dispatched before it must also have been retired. Hence, instructions may complete execution out of order, but they are retired in program order.

6.9.4 DISPATCH OPERATION

We now return to the dispatch operation. When dispatching decisions are made, the dispatch unit must ensure that all the resources needed for the execution of an instruction are available. For example, since the results of an instruction may have to be written in a temporary register, there should be one available, and it is reserved for use by that instruction as a part of the dispatch operation. There must be space available in the reservation station of an appropriate execution unit. Finally, a location in the reorder buffer for later commitment of results must also be available for the instruction. When all the resources needed are assigned, the instruction is dispatched.

Should instructions be dispatched out of order? For example, the dispatch of the Load instruction in Figure 6.14 may be delayed because there is no space in the reservation station of the Load/Store unit as a result of a cache miss in a previously dispatched instruction. Should the Subtract instruction be dispatched instead? In principle this is possible, provided that all the resources needed by the Load instruction, including a place in the reorder buffer, are reserved for it. This is essential to ensure that all instructions are ultimately retired in the correct order and that no deadlocks occur.

A *deadlock* is a situation that can arise when two units, A and B, use a shared resource. Suppose that unit B cannot complete its operation until unit A completes its operation. At the same time, unit B has been assigned a resource that unit A needs. If this happens, neither unit can complete its operation. Unit A is waiting for the resource it needs, which is being held by unit B. At the same time, unit B is waiting for unit A to finish before it can complete its operation and release that resource.

As an example of a deadlock when dispatching instructions out of order, consider a superscalar processor that has only one temporary register. When the Subtract instruction in Figure 6.14 is dispatched before the Load instruction, the temporary register is reserved for it. The Load instruction cannot be dispatched because it is waiting for the same temporary register, which, in turn, will not become free until the Subtract instruction is retired. Since the Subtract instruction cannot be retired before the Load instruction, we have a deadlock.

To prevent deadlocks, the dispatch unit must take many factors into account. Hence, issuing instructions out of order is likely to increase the complexity of the dispatch unit significantly. It may also mean that more time is required to make dispatching decisions. Dispatching instructions in order avoids this complexity. In this case, the program order of instructions is enforced at the time instructions are dispatched and again at the time they are retired. Between these two events, the execution of several instructions across multiple execution units can proceed out of order, subject only to interdependencies among them.

A final comment on superscalar processors concerns the number of execution units. The processor in Figure 6.13 has one arithmetic unit and one Load/Store unit. For higher performance, modern superscalar processors often have two arithmetic units for integer operations, as well as a separate arithmetic unit for floating-point operations. The floating-

point unit has its own register file. Many processors also include a vector unit for integer or floating-point arithmetic, which typically performs two to eight operations in parallel. Such a unit may also have a dedicated register file. A single Load/Store unit typically supports all memory accesses to or from the register files for integer, floating-point, or vector units. To keep many execution units busy, modern processors may fetch four or more instructions at the same time to place at the tail of the instruction queue, and similarly four or more instructions may be dispatched to the execution units from the head of the instruction queue.

6.10 PIPELINING IN CISC PROCESSORS

The instruction set of a RISC processor makes pipelining relatively easy to implement. All instructions are one word in size, and operand information is typically located in the same position within a word for different instructions. No instruction requires more than one memory operand. Only Load and Store instructions access memory operands, typically using only indexed addressing. All other instructions operate on register operands. The five-stage pipeline described in this chapter is tailored for these characteristics of RISC-style instructions.

For pipelining in CISC processors, complications arise due to instructions that are variable in size, have multiple memory operands and complex addressing modes, and use condition codes. Instructions that occupy more than one word may take several cycles to fetch. Furthermore, variability in instruction size and format complicates both decoding and operand access, as well as management of the dispatch queue in a superscalar processor.

The availability of more complex addressing modes such as Autoincrement or Autodecrement introduces *side effects* when executing instructions. A side effect occurs when a location other than that of the destination operand is also affected. For example, the instruction

```
Move    R5, (R8)+
```

has a side effect. Not only is the destination register R5 affected, but source register R8 is also affected by the autoincrement operation. Should a later instruction depend on the value in register R8, this dependency must be handled with additional hardware in the same manner as a dependency involving the destination register, R5. It may require stalling the pipeline or forwarding the new value. In a superscalar processor, such a dependency requires the use of temporary registers and register renaming as discussed in Section 6.9.3.

Condition codes also introduce side effects. For example, in the sequence of instructions

```
Compare     R7, R8
Branch>0    TARGET
```

the result of the Compare instruction affects the condition code flags as a side effect. The Branch instruction, in turn, implicitly depends on this side effect. A condition code register can be included with relative ease in a simple pipeline such as the one shown in Figure 6.2, because only one ALU operation is performed in any cycle. However, in a superscalar processor with multiple execution units, many instructions may be in various

stages of execution, and two or more ALU operations may be performed in each cycle. Dependencies arising from side effects related to the condition codes require the use of additional temporary registers and register renaming.

Finally, consider the following sequence of CISC-style instructions:

```
Move    (R2), (R3)
Move    (R4), R5
```

The first Move instruction requires two operand accesses to the memory, while the second Move instruction requires only one. Executing these instructions in a pipeline such as the one in Figure 6.2 requires additional hardware to stall the second Move instruction so that the first Move instruction can complete its two operand accesses to the memory. In a superscalar processor such as the one in Figure 6.13, the Load/Store unit must similarly stall its internal pipeline.

CISC-style instructions complicate pipelining. This was one of the main reasons for developing the RISC approach. Nonetheless, pipelined processors have been implemented for CISC-style instruction sets, which were initially introduced before the widespread use of pipelining. Examples include processors based on the ColdFire and Intel instruction sets discussed in Appendices C and E. ColdFire processors are primarily intended for embedded applications, while Intel processors serve general-purpose needs. Consequently, the extent to which pipelining is used in ColdFire processors is less than that in Intel processors.

6.10.1 PIPELINING IN COLDFIRE PROCESSORS

ColdFire processor implementations labeled as versions V1 and V2 have two pipelines in series with a first-in first-out (FIFO) buffer between them. A two-stage instruction fetch pipeline prefetches instructions into the buffer. This buffer then feeds a two-stage pipeline that executes instructions. Instructions that involve register-only or register-to-memory operations pass once through the two execution stages. Instructions that involve memory-to-register or memory-to-memory operations must make two passes through the execution stages.

Later versions of ColdFire processor implementations use a similar buffer arrangement between two pipelines, but they incorporate various enhancements for higher performance. For example, the instruction fetch pipeline in version V4 is extended to four stages and includes branch prediction. The execution pipeline is extended to five stages. The early stages are used for address calculation, and the later stages are used for arithmetic/logic operations. This separation of functions enables a limited form of superscalar processing. In certain cases, a Move instruction and another instruction can be issued to the execution pipeline in the same cycle. Version V5 implementations have two distinct execution pipelines based on the V4 organization. They provide true superscalar processing.

6.10.2 PIPELINING IN INTEL PROCESSORS

Intel processors achieve high performance with superscalar execution and deep pipelines. For example, the Core 2 and Core i7 architectures have a multiple-issue width of four

instructions and a 14-stage pipeline. Branch prediction, register renaming, out-of-order execution, and other techniques are used.

To reduce internal complexity, CISC-style instructions are dynamically converted by the hardware into simpler RISC-style micro-operations. These micro-operations are then issued to the execution units to complete the tasks specified by the original CISC-style instructions. This approach preserves code compatibility while making it possible to use the aggressive performance enhancement techniques that have been developed for RISC-style instruction sets. In some cases, micro-operations are fused back together into macro-operations for more efficient handling. For example, in a program containing original CISC-style instructions, a comparison instruction that affects condition codes is often followed by a branch instruction. The hardware may initially convert the comparison and branch instructions into separate micro-operations, but would then fuse them into a combined compare-and-branch operation, whose function reflects what is typically found in a RISC-style instruction set.

6.11 CONCLUDING REMARKS

Two important features for performance enhancement have been introduced in this chapter, pipelining and multiple-issue. Pipelining enables processors to have instruction throughput approaching one instruction per clock cycle. Multiple-issue combined with pipelining makes possible superscalar operation, with instruction throughput of several instructions per clock cycle.

The potential gain in performance can only be realized by careful attention to three aspects:

- The instruction set of the processor
- The design of the pipeline hardware
- The design of the associated compiler

It is important to appreciate that there are strong interactions among all three aspects. High performance is critically dependent on the extent to which these interactions are taken into account in the design of a processor. Instruction sets that are particularly well-suited for pipelined execution are key features of modern processors.

There are many sources that provide additional details on the topics presented in this chapter. Reference [1] covers pipelining and Reference [2] covers superscalar processors.

6.12 EXAMPLES OF SOLVED PROBLEMS

This section presents some examples of the types of problems that a student may be asked to solve, and shows how such problems can be solved.

Example 6.1

Problem: Consider the pipelined execution of the following sequence of instructions:

```
Add       R4, R3, R2
Or        R7, R6, R5
Subtract  R8, R7, R4
```

Initially, registers R2 and R3 contain 4 and 8, respectively. Registers R5 and R6 contain 128 and 2, respectively. Assume that the pipeline provides forwarding paths to the ALU from registers RY and RZ in Figure 5.8. The first instruction is fetched in cycle 1, and the remaining instructions are fetched in successive cycles.

Draw a diagram similar to Figure 6.1 to show the pipelined execution of these instructions assuming that the processor uses operand forwarding. Then, with reference to Figure 5.8, describe the contents of registers RY and RZ during cycles 4 to 7.

Solution: There are data dependencies involving registers R4 and R7. The Subtract instruction needs the new values for these registers before they are written to the register file. Hence, those values need to be forwarded to the ALU inputs when the Subtract instruction is in the Compute stage of the pipeline. Figure 6.15 shows the execution with forwarding. One arrow represents the new value of register R7 being forwarded from register RZ, and the other arrow represents the new value of register R4 being forwarded from register RY.

As for the contents of registers RY and RZ during cycles 4 to 7, the following description provides the answer.

- Using the initial values for registers R2 and R3, the Add instruction generates the result of 12 in cycle 3. That result is available in register RZ during cycle 4. The value in register RY during cycle 4 is the result for the unspecified instruction preceding the Add instruction.
- In cycle 4, the Or instruction generates the result of 130. That result is placed in register RZ to be available during cycle 5. The result of 12 for the Add instruction is in register RY during cycle 5.
- In cycle 5, the Subtract instruction is in the Compute stage. To generate a correct result, forwarding is used to provide the value of 130 in register RY and the value of

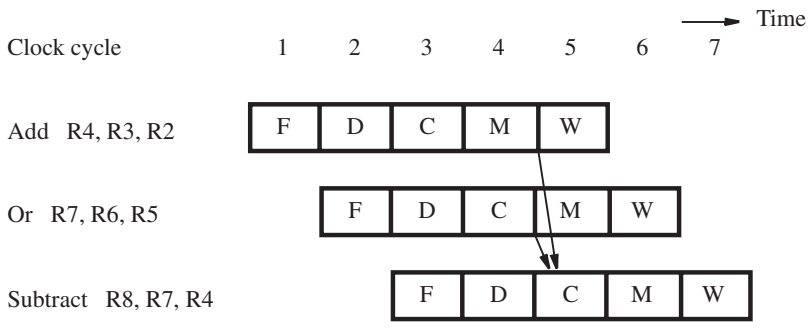


Figure 6.15 Pipelined execution of instructions for Example 6.1.

12 in register RZ. The result from the ALU is $130 - 12 = 118$. This result is available in register RZ during cycle 6. The result of the Or instruction, 130, is in register RY during in cycle 6.

- In cycle 6, the Subtract instruction is in the Memory stage. The unspecified instruction following the Subtract instruction is generating a result in the Compute stage. In cycle 7, the result of the unspecified instruction is in register RZ, and the result of the Subtract instruction is in register RY.

Example 6.2 **Problem:** Assume that 20 percent of the dynamic count of the instructions executed for a program are branch instructions. There are no pipeline stalls due to data dependencies. Static branch prediction is used with a not-taken assumption.

(*a*) Determine the execution times for two cases: when 30 percent of the branches are taken, and when 70 percent of the branches are taken.

(*b*) Determine the speedup for one case relative to the other. Express the speedup as a percentage relative to 1.

Solution: Section 6.8.1 describes the calculation of $\delta_{\text{branch\_penalty}}$ to consider the effect of branch penalties.

(*a*) The value of $\delta_{\text{branch\_penalty}}$ is $0.20 \times 0.30 = 0.06$ for the first case and $0.20 \times 0.70 = 0.14$ for the second case. Using $S = 1 + \delta_{\text{branch\_penalty}}$, the execution time for the first case is $(1.06 \times N)/R$ and $(1.14 \times N)/R$ for the second case.

(*b*) Because the execution time for the first case is smaller, the performance improvement as a speedup percentage is

$$\left(\frac{1.14}{1.06} - 1\right) \times 100 = 7.5 \text{ percent}$$

PROBLEMS

6.1 **[M]** Consider the following instructions at the given addresses in the memory:

1000	Add	R3, R2, #20
1004	Subtract	R5, R4, #3
1008	And	R6, R4, #0x3A
1012	Add	R7, R2, R4

Initially, registers R2 and R4 contain 2000 and 50, respectively. These instructions are executed in a computer that has a five-stage pipeline as shown in Figure 6.2. The first instruction is fetched in clock cycle 1, and the remaining instructions are fetched in successive cycles.

(*a*) Draw a diagram similar to Figure 6.1 that represents the flow of the instructions through the pipeline. Describe the operation being performed by each pipeline stage during clock cycles 1 through 8.

(*b*) With reference to Figures 5.8 and 5.9, describe the contents of registers IR, PC, RA, RB, RY, and RZ in the pipeline during cycles 2 to 8.

6.2 **[M]** Repeat Problem 6.1 for the following program:

```
1000    Add         R3, R2, #20
1004    Subtract    R5, R4, #3
1008    And         R6, R3, #0x3A
1012    Add         R7, R2, R4
```

Assume that the pipeline provides forwarding paths to the ALU from registers RY and RZ in Figure 5.8 and that the processor uses forwarding of operands.

6.3 **[M]** Consider the loop in the program of Figure 2.8. Assume it is executed in a five-stage pipeline with forwarding paths to the ALU from registers RY and RZ in Figure 5.8. Assume that the pipeline uses static branch prediction with a not-taken assumption. Draw a diagram similar to Figure 6.1 for the execution of two successive iterations of the loop.

6.4 **[D]** Repeat Problem 6.3, but first reorder the instructions to optimize performance as the compiler would do.

6.5 **[D]** Repeat Problem 6.3 for a pipeline that uses delayed branching with one delay slot. Reorder the instructions as needed to improve performance.

6.6 **[M]** The forwarding path in Figure 6.5 allows the contents of register RZ to be used directly in an ALU operation. The result of that operation is stored in register RZ, replacing its previous contents. This problem involves tracing the contents of register RZ over multiple cycles. Consider the two instructions

```
I1:   Add       R3, R2, R1
I2:   LShiftL   R3, R3, #1
```

While instruction I_1 is being fetched in cycle 1, a previously fetched instruction is performing an ALU operation that gives a result of 17. Then, while instruction I_1 is being decoded in cycle 2, another previously fetched instruction is performing an ALU operation that gives a result of 198. Also during cycle 2, registers R1, R2, and R3 contain the values 30, 100, and 45, respectively. Using this information, draw a timing diagram that shows the contents of register RZ during cycles 2 to 5.

6.7 **[M]** Assume that 20 percent of the dynamic count of the instructions executed for a program are branch instructions. Delayed branching is used, with one delay slot. Assume that there are no stalls caused by other factors. First, derive an expression for the execution time in

cycles if all delay slots are filled with NOP instructions. Then, derive another expression that reflects the execution time with 70 percent of delay slots filled with useful instructions by the optimizing compiler. From these expressions, determine the compiler's contribution to the increase in performance, expressed as a speedup percentage.

6.8 **[D]** Repeat Problem 6.7, but this time for a pipelined processor with two branch delay slots. The output from the optimizing compiler is such that the first delay slot is filled with a useful instruction 70 percent of the time, but the second slot is filled with a useful instruction only 10 percent of the time.

Compare the compiler-optimized execution time for this case with the compiler-optimized execution time for Problem 6.7. Assume that the two processors have the same clock rate. Indicate which processor/compiler combination is faster, and determine the speedup percentage by which it is faster.

6.9 **[D]** Assume that 20 percent of the dynamic count of the instructions executed for a program are branch instructions. Assume further that 75 percent of branches are actually taken. The program is executed in two different processors that have the same clock rate. One uses static branch prediction with the assume-not-taken approach. The other uses dynamic branch prediction based on the states in Figure 6.12*a*. The branch target buffer is used in the manner described in Section 6.6.4.

(*a*) With no pipeline stalls due to other causes, what must be the minimum prediction accuracy for the processor using dynamic branch prediction to perform at least as well as the processor using static branch prediction?

(*b*) If the dynamic prediction accuracy is actually 90 percent, what is the speedup relative to using static prediction?

6.10 **[M]** Additional control logic is required in the pipeline to forward the value of register RZ as shown in Figure 6.5. What specific conditions must this additional logic check to determine the settings of the multiplexers feeding the ALU inputs in the Compute stage of the pipeline?

6.11 **[M]** Repeat Problem 6.10 for the specific conditions related to forwarding of the contents of register RY in Figure 5.8 to the multiplexers feeding the inputs of the ALU.

6.12 **[D]** As a continuation of Problems 6.10 and 6.11, consider the following sequence of instructions:

```
Add       R3, R2, R1
Subtract  R3, R5, R4
Or        R8, R3, #1
```

Describe the manner in which forwarding must be handled for this situation. How should the conditions developed in Problems 6.10 and 6.11 be modified?

6.13 **[M]** Consider a program that consists of four memory-access instructions and four arithmetic instructions. Assume that there are no data dependencies between the instructions. Two versions of this program are executed on the superscalar processor shown in Figure 6.13. The first version has the four memory-access instructions in sequence, followed by the four arithmetic instructions. The second version has the memory-access instructions

interleaved with the arithmetic instructions. Draw two diagrams similar to Figure 6.14 to compare the execution of these two versions of the program.

6.14 **[E]** Assume that a program contains no branch instructions. It is executed on the superscalar processor shown in Figure 6.13. What is the best execution time in cycles that can be expected if the mix of instructions consists of 75 percent arithmetic instructions and 25 percent memory-access instructions? How does this time compare to the best execution time on the simpler processor in Figure 6.2 using the same clock?

6.15 **[M]** Repeat Problem 6.14 to find the best possible execution times for the processors in Figures 6.2 and 6.13, assuming that the mix of instructions consists of 15 percent branch instructions that are never taken, 65 percent arithmetic instructions, and 20 percent memory-access instructions. Assume a prediction accuracy of 100 percent for all branch instructions.

6.16 **[E]** Consider a processor that uses the branch prediction scheme represented in Figure 6.12*b*. The instruction set for the processor is enhanced with a feature that enables the compiler to specify the initial prediction state as either LT or LNT for each branch instruction. This initial state is used by the processor at execution time when information about the branch instruction is not found in the branch target buffer. Discuss how the compiler should use this feature when generating code for the following cases:

(*a*) A loop with a conditional branch instruction at the end to branch to the start of the loop

(*b*) A loop with a conditional branch at the beginning of the loop to exit the loop, and an unconditional branch at the end of the loop to branch to the start

6.17 **[M]** Assume that a processor has the feature described in Problem 6.16 for specifying the initial prediction state for branch instructions. Consider a statement of the form

IF A>B THEN A = A + 1 ELSE B = B + 1

(*a*) Generate assembly-language code for the statement above.

(*b*) In the absence of any other information, discuss how the compiler should specify the initial prediction state for the branch instructions in the assembly code.

(*c*) A study of the execution behavior of the program containing the above statement reveals that the value of variable A is often larger than the value of variable B. If this information is made available to the compiler, discuss how it would influence the initial prediction state for the branch instructions.

6.18 **[M]** Consider a statement of the form

IF A>B THEN A = A + 1 ELSE B = B + 1

(*a*) Consider a processor that has the pipelined organization shown in Figure 6.2, with static branch prediction that uses a not-taken assumption. Write assembly-language code for the statement above. Draw diagrams similar to Figure 6.1 to show the pipelined execution of the instructions for different branch decisions and determine the execution times in cycles.

(*b*) Now assume that delayed branching is used. Write assembly-language code for the statement above. Draw diagrams to show the pipelined execution of the instructions for different branch decisions and compare the execution times in cycles with the times for the previous case.

REFERENCES

1. D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 4th edition, Morgan Kaufmann, Burlington, Massachusetts, 2009.
2. J. P. Shen and M. H. Lipasti, *Modern Processor Design: Fundamentals of Superscalar Processors*, McGraw-Hill, New York, 2005.